

NOÇÕES DE COMPLEXIDADE DE ALGORITMOS

DCE792 – Algoritmos e Estruturas de Dados II (Prática)

Atualizado em: 11 de agosto de 2026

Iago Carvalho

Departamento de Ciência da Computação



O ALGORITMO FUNCIONA. MAS ELA ESCALA?

Mudança de perspectiva

No início, a pergunta é: “COMO RESOLVER?”

Com experiência, surge outra: “ESTA SOLUÇÃO CONTINUA VIÁVEL QUANDO A ENTRADA CRESCE?”

- Uma solução rápida para 100 itens pode travar com 10 milhões
- Mais hardware ajuda, mas não corrige qualquer taxa de crescimento
- Analisar complexidade permite comparar ideias antes de implementá-las

PROBLEMA, INSTÂNCIA E ALGORITMO

Problema: Tarefa geral a resolver: ordenar, buscar, conectar, otimizar

Instância: Um caso concreto do problema: o vetor $[8, 3, 1, 6]$

Algoritmo: Sequência finita e bem definida de passos que transforma entrada em saída

Tamanho: Medida da entrada, normalmente representada por n

Exemplo: ordenação

Entrada: uma sequência de n valores

Saída: os mesmos valores em ordem crescente

POR QUE NÃO MEDIR APENAS EM SEGUNDOS?

O tempo observado depende de:

- processador, memória, disco e carga da máquina;
- linguagem, compilador, otimizações e sistema operacional;
- implementação e dados usados no teste.

Em vez de cronometrar uma máquina específica, estimamos o **NÚMERO DE OPERAÇÕES RELEVANTES** em função do tamanho da entrada

QUAIS RECURSOS PODEMOS ANALISAR?

Tempo

Quantidade de instruções ou operações fundamentais executadas.

Exemplos: comparações, atribuições, acessos, chamadas.

Espaço

Quantidade de memória adicional necessária durante a execução.

Exemplos: vetores auxiliares, tabelas, pilha de recursão.

Complexidade não é só velocidade

Em sistemas reais, também podem importar acessos a disco, rede, energia e latência, dentre outros aspectos

ESCOLHENDO O TAMANHO DA ENTRADA

O símbolo n precisa representar o aspecto que faz o trabalho crescer.

Problema	Tamanho relevante
Buscar em vetor	número de elementos
Multiplicar matrizes	linhas e colunas
Processar texto	número de caracteres
Percorrer grafo	vértices V e arestas E
Counting sort	itens n e faixa de chaves k

Atenção

Podem existir uma, duas, ou mais variáveis de entrada

MODELO DE CUSTO: OPERAÇÕES PRIMITIVAS

Para uma primeira análise, tratamos como custo constante:

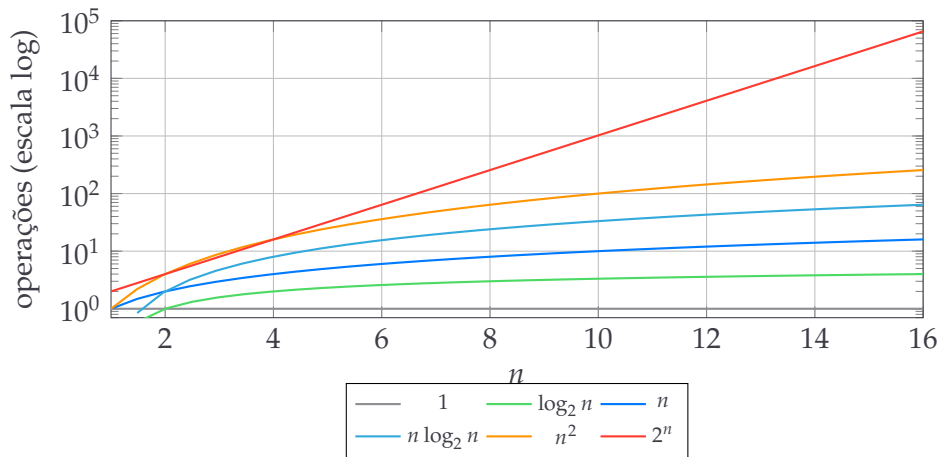
- atribuições e comparações simples;
- operações aritméticas em tipos de tamanho fixo;
- acesso a uma posição de vetor;
- incremento de contador e retorno de função.

CONTANDO OPERAÇÕES: BUSCA DO MENOR

```
1 int menor = v[0];  
2 for (int i = 1; i < n; i++) {  
3     if (v[i] < menor) {  
4         menor = v[i];  
5     }  
6 }
```

- A inicialização ocorre uma vez.
- O laço faz $n - 1$ comparações.
- A atribuição interna depende dos dados: entre 0 e $n - 1$ vezes.

FAMÍLIAS DE CRESCIMENTO



Dizemos que $T(n) = O(g(n))$ se existem constantes $c > 0$ e n_0 tais que

$$0 \leq T(n) \leq c g(n), \quad \text{para todo } n \geq n_0.$$

Em outras palavras...

A partir de algum tamanho, $T(n)$ cresce no máximo tão rápido quanto $g(n)$, a menos de uma constante

TEMPO CONSTANTE: $O(1)$

```
1 int primeiro(const int v[]) {  
2     return v[0];  
3 }
```

- O número de operações não depende do número de elementos.
- $O(3)$, $O(50)$ e qualquer custo fixo são escritos como $O(1)$.
- Constante não significa “instantâneo”; significa “não cresce com n ”.

Exemplos: acesso por índice em vetor, empilhar em pilha dinâmica sem realocação, consulta média em hash bem dimensionado.

TEMPO LOGARÍTMICO: $O(\log n)$

Aparece quando cada passo reduz o problema por um fator constante.

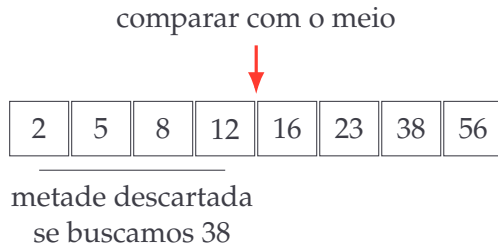
$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1$$

Após k divisões por 2, $n/2^k = 1$, portanto $k = \log_2 n$.

n	8	1.024	1.048.576	2^{40}
$\log_2 n$	3	10	20	40

Dobrar a entrada acrescenta apenas um passo.

BUSCA BINÁRIA: DESCARTANDO METADE



- Requisito: a coleção deve estar ordenada e permitir acesso eficiente ao meio.
- Pior caso: no máximo $\lfloor \log_2 n \rfloor + 1$ comparações.

BUSCA BINÁRIA EM C

```
1 int busca_binaria(int v[], int n, int x) {  
2     int esq = 0, dir = n - 1;  
3     while (esq <= dir) {  
4         int meio = esq + ((dir - esq) / 2);  
5         if (v[meio] == x) {  
6             return meio;  
7         } else if (v[meio] < x) {  
8             esq = meio + 1;  
9         } else {  
10            dir = meio - 1;  
11        }  
12    }  
13    return -1;  
14 }
```

O intervalo de busca cai aproximadamente pela metade a cada iteração

TEMPO LINEAR: $O(n)$

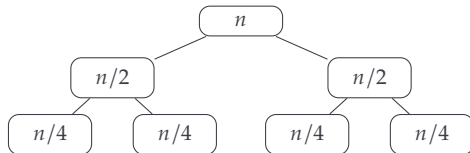
```
1 int soma(const int v[], int n) {  
2     int total = 0;  
3     for (int i = 0; i < n; i++) {  
4         total += v[i];  
5     }  
6     return total;  
7 }
```

- Cada elemento é visitado uma vez.
- Se a entrada dobra, o trabalho aproximadamente dobra.
- Dois laços consecutivos de n iterações dão $2n$, portanto $O(n)$.

TEMPO LINEARÍTMICO: $O(n \log n)$

É comum em algoritmos eficientes de ordenação por comparação:

- dividir a entrada em níveis: cerca de $\log_2 n$ níveis;
- processar, em cada nível, um total de cerca de n elementos;
- custo total: n por nível $\times \log n$ níveis.



Exemplos: mergesort; heapsort; quicksort

TEMPO QUADRÁTICO: $O(n^2)$

```
1 for (int i = 0; i < n; i++) {  
2     for (int j = 0; j < n; j++) {  
3         compara(v[i], v[j]);  
4     }  
5 }
```

- Para cada uma das n iterações externas, há n internas.
- Total: $n \cdot n = n^2$ execuções de processa.
- Se n multiplica por 10, o trabalho multiplica aproximadamente por 100.

Exemplo: comparar todos os possíveis pares de elementos de uma lista

LAÇO TRIANGULAR AINDA É QUADRÁTICO

```
1 for (int i = 0; i < n-1; i++) {  
2     for (int j = i+1; j < n; j++) {  
3         compara(v[i], v[j]);  
4     }  
5 }
```

Se o laço interno começa em $i + 1$, o número de pares é

$$(n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2}.$$

Então:

$$T(n) = \frac{n^2 - n}{2} = O(n^2).$$

TEMPO CÚBICO: $O(n^3)$

```
1 for (int i = 0; i < n; i++) {  
2     for (int j = 0; j < n; j++) {  
3         for (int k = 0; k < n; k++) {  
4             usa(i, j, k);  
5         }  
6     }  
7 }
```

Três dimensões independentes de tamanho n geram n^3 combinações.

n	10	100	1.000
n^3	1.000	1.000.000	1.000.000.000

MULTIPLICAÇÃO DE MATRIZES

Para matrizes quadradas A e B de ordem n por n :

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}.$$

- Há n^2 posições em C .
- Cada posição exige n multiplicações no algoritmo clássico.
- Tempo: $O(n^3)$; espaço da saída: $O(n^2)$.

Conte a operação fundamental

Neste exemplo, multiplicar dois elementos é uma boa operação para acompanhar.

TEMPO EXPONENCIAL: $O(2^n)$

```
1 long fib(int n) {  
2     if (n <= 1) {  
3         return n;  
4     }  
5     return fib(n - 1) + fib(n - 2);  
6 }
```

A árvore de chamadas repete muitos subproblemas.

- O número de chamadas cresce exponencialmente.
- Memorizar resultados reduz o tempo para $O(n)$, usando $O(n)$ espaço.
- Crescimentos exponenciais tornam-se inviáveis rapidamente.

TEMPO FATORIAL: $O(n!)$

Surge quando enumeramos todas as permutações de n elementos.

n	5	10	15	20
$n!$	120	3.628.800	$1,31 \times 10^{12}$	$2,43 \times 10^{18}$

Exemplo

Testar todas as ordens de visita de cidades em uma solução ingênua para o caixeiro-viajante.

Algoritmos exatos ainda podem ser úteis quando n é pequeno ou quando há boas podas.

HIERARQUIA DE CRESCIMENTO

Para entradas grandes, em ordem crescente:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!).$$

A ordem é assintótica

Para entradas pequenas, constantes, implementação e hardware ainda podem inverter medições práticas.

UMA COMPARAÇÃO NUMÉRICA

Valores aproximados para $n = 1.000$:

Função	Número de operações
$\log_2 n$	10
n	1.000
$n \log_2 n$	10.000
n^2	1.000.000
n^3	1.000.000.000
2^n	mais de 10^{300}
$n!$	cerca de $4,02 \times 10^{2567}$

A taxa de crescimento costuma importar mais que pequenas otimizações locais.

COMPLEXIDADE E TEMPO DE EXECUÇÃO

Função de complexidade	n (tamanho do problema)		
	20	40	60
n	0,0002 s	0,0004 s	0,0006 s
$n \log_2 n$	0,0009 s	0,0021 s	0,0035 s
n^2	0,0040 s	0,0160 s	0,0360 s
n^3	0,0800 s	0,6400 s	2,1600 s
2^n	10,0000 s	27 dias	3.660 séculos
3^n	580 minutos	38.550 séculos	$1,3 \times 10^{14}$ séculos
$n!$	7.700 séculos	$2,6 \times 10^{33}$ séculos	$2,6 \times 10^{67}$ séculos

Valores aproximados, supondo 10 microssegundos por operação.

O QUE UMA MÁQUINA MAIS RÁPIDA CONSEGUE RESOLVER?

Função de complexidade	Maior instância solucionável em uma hora		
	máquina original	máquina 100 vezes mais rápida	máquina 1.000 vezes mais rápida
n	N	$100N$	$1.000N$
$n \log_2 n$	N_1	$22,5N_1$	$140,2N_1$
n^2	N_2	$10N_2$	$31,6N_2$
n^3	N_3	$4,6N_3$	$10N_3$
2^n	N_4	$N_4 + 6$	$N_4 + 10$
3^n	N_5	$N_5 + 4$	$N_5 + 6$
$n!$	N_6	$N_6 + 2$	$N_6 + 3$

Efeito sobre a complexidade fatorial

No modelo de 10 microssegundos por operação, $N_6 = 11$. Em uma hora, as máquinas 100 e 1.000 vezes mais rápidas alcançam apenas $n = 13$ e $n = 14$, respectivamente.

BLOCOS SEQUENCIAIS: SOMAR CUSTOS

```
1 for (int i = 0; i < n; i++) tarefa_a(i); // O(n)
2 for (int i = 0; i < n; i++) tarefa_b(i); // O(n)
3 for (int i = 0; i < n*n; i++) tarefa_c(i); // O(n^2)
```

$$O(n) + O(n) + O(n^2) = O(n^2).$$

Se houver laços em sequência, some os custos e conserve o termo dominante

BLOCOS ANINHADOS: MULTIPLICAR CUSTOS

```
1 for (int i = 0; i < n; i++) {           // n vezes
2     for (int j = 1; j < n; j *= 2) {    // log n vezes
3         tarefa(i, j);                  // O(1)
4     }
5 }
```

$$n \cdot \log_2 n \cdot O(1) = O(n \log n) .$$

Não conte apenas o número de laços: observe quantas vezes cada um executa

CONDICIONAIS: DEPENDE DO CASO

```
1 if (condicao) {  
2     algoritmo_linear(v, n);      // O(n)  
3 } else {  
4     algoritmo_quadratico(v, n);   // O(n^2)  
5 }
```

- Pior caso: $\max(O(n), O(n^2)) = O(n^2)$.
- Melhor caso possível: $O(n)$, se o ramo linear puder ocorrer.
- Caso médio: precisa da probabilidade de cada ramo e de seus custos.

ENTRADAS INDEPENDENTES: n E m

```
1 for (int i = 0; i < n; i++) usa(a[i]);  
2 for (int j = 0; j < m; j++) usa(b[j]);
```

Custo: $O(n + m)$.

```
1 for (int i = 0; i < n; i++)  
2     for (int j = 0; j < m; j++)  
3         combina(a[i], b[j]);
```

Custo: $O(nm)$.

Regra

Use a mesma variável apenas quando os tamanhos realmente estiverem vinculados.

ESPAÇO TOTAL E ESPAÇO AUXILIAR

Espaço total: Entrada, saída e memória temporária.

Espaço auxiliar: Memória extra além da entrada e da saída exigida.

Exemplos

- Somar um vetor com uma variável acumuladora: $O(1)$ auxiliar.
- Copiar um vetor de n posições: $O(n)$ auxiliar.
- Tabela $n \times n$: $O(n^2)$.

Sempre declare qual noção de espaço está sendo usada.

PROBLEMA DOS DOIS VALORES

Entrada

Um vetor de n inteiros e um alvo s .

Saída

Dois índices $i \neq j$ tais que $v[i] + v[j] = s$, se existirem.

Exemplo: $v = [7, 2, 11, 15]$ e $s = 9$ tem como resposta $i = 7$ e $j = 2$

Pergunta

Como resolver? E qual é o custo da solução?

SOLUÇÃO: TESTAR TODOS OS PARES

```
1 for (int i = 0; i < n; i++) {  
2     for (int j = i + 1; j < n; j++) {  
3         if (v[i] + v[j] == alvo)  
4             return ENCONTROU;  
5     }  
6 }
```

- Pior caso: testa $n(n - 1)/2$ pares.
- Tempo: $O(n^2)$.
- Espaço auxiliar: $O(1)$.

É simples, correto e pode ser suficiente para entradas pequenas.

CUSTOS TÍPICOS DE ESTRUTURAS

Valores médios ou amortizados em implementações usuais:

Estrutura/operação	Acesso ou busca	Inserção	Remoção
Vetor por índice	$O(1)$	–	–
Vetor não ordenado por valor	$O(n)$	$O(1)$	$O(n)$
Lista encadeada por valor	$O(n)$	$O(1)$	$O(1)$
Árvore balanceada	$O(\log n)$	$O(\log n)$	$O(\log n)$
Tabela hash	$O(1)$ esperado	$O(1)$ esperado	$O(1)$ esperado

A representação dos dados altera os algoritmos disponíveis e seus custos.

Análise assintótica

Explica como o custo cresce e permite generalizar entre máquinas.

Medição experimental

Revela constantes, cache, compilador, alocação, E/S e dados reais.

1. Analise para eliminar estratégias que não escalam.
2. Meça implementações plausíveis no cenário representativo.
3. Otimize apenas gargalos confirmados, sem perder correção.

ARMADILHAS FREQUENTES

- Confundir $O(n)$ com um tempo exato de n segundos.
- Dizer que Big O sempre representa o pior caso.
- Contar laços sem observar seus limites e atualizações.
- Somar laços aninhados, em vez de multiplicar seus custos.
- Trocar $O(n + m)$ por $O(n)$ quando n e m são independentes.
- Ignorar ordenação prévia, distribuição dos dados ou custo da hash.
- Esquecer memória auxiliar e profundidade da recursão.
- Escolher algoritmo só pela notação e ignorar o domínio real.

ATIVIDADE 1: CLASSIFIQUE OS TRECHOS

Determine tempo e espaço auxiliar.

```
1 // A
2 for (int i = 1; i < n; i *= 2) usa(i);
3
4 // B
5 for (int i = 0; i < n; i++)
6     for (int j = 0; j < 100; j++) usa(i, j);
7
8 // C
9 for (int i = 0; i < n; i++)
10     for (int j = i + 1; j < n; j++) usa(i, j);
```

ATIVIDADE 1: CLASSIFIQUE OS TRECHOS

Determine tempo e espaço auxiliar.

```
1 // A
2 for (int i = 1; i < n; i *= 2) usa(i);
3
4 // B
5 for (int i = 0; i < n; i++)
6     for (int j = 0; j < 100; j++) usa(i, j);
7
8 // C
9 for (int i = 0; i < n; i++)
10     for (int j = i + 1; j < n; j++) usa(i, j);
```

Respostas

A: $\Theta(\log n)$; B: $\Theta(n)$; C: $\Theta(n^2)$. Todos usam $\Theta(1)$ de espaço auxiliar.

ATIVIDADE 2: UM CASO MENOS ÓBVIO

```
1 for (int i = 0; i < n; i++) {  
2     int j = n;  
3     while (j > 1) {  
4         j /= 2;  
5         usa(i, j);  
6     }  
7 }
```

1. Quantas vezes executa o `for`?
2. Quantas vezes executa o `while` para cada i ?
3. Qual é o produto dos custos?

ATIVIDADE 2: UM CASO MENOS ÓBVIO

```
1 for (int i = 0; i < n; i++) {  
2     int j = n;  
3     while (j > 1) {  
4         j /= 2;  
5         usa(i, j);  
6     }  
7 }
```

1. Quantas vezes executa o for?
2. Quantas vezes executa o while para cada i ?
3. Qual é o produto dos custos?

Resposta

n iterações externas e $\lfloor \log_2 n \rfloor$ internas: $\Theta(n \log n)$ no tempo e $\Theta(1)$ no espaço auxiliar.

ATIVIDADE 3: ESCOLHA DE ALGORITMO

Uma aplicação realiza muitas buscas em uma coleção estática com n elementos.

- Alternativa A: busca sequencial em $O(n)$ por consulta.
- Alternativa B: ordenar uma vez em $O(n \log n)$ e usar busca binária em $O(\log n)$ por consulta.

Para q consultas:

$$A = O(qn), \quad B = O(n \log n + q \log n).$$

Discussão

Quando o custo inicial de ordenar começa a compensar? O que muda se houver muitas inserções?

TESTE RÁPIDO

1. Se um algoritmo tem complexidade $20n^2 + 500n + 3$, qual é a classe dominante?
2. Dois laços consecutivos de n passos formam $O(n^2)$?
3. Um laço que divide n por 2 a cada passo é de qual ordem?
4. Acesso por índice em vetor é tipicamente de qual ordem?
5. Uma solução com melhor Big O é sempre mais rápida para qualquer entrada?

TESTE RÁPIDO

1. Se um algoritmo tem complexidade $20n^2 + 500n + 3$, qual é a classe dominante?
2. Dois laços consecutivos de n passos formam $O(n^2)$?
3. Um laço que divide n por 2 a cada passo é de qual ordem?
4. Acesso por índice em vetor é tipicamente de qual ordem?
5. Uma solução com melhor Big O é sempre mais rápida para qualquer entrada?

Gabarito

- 1) $O(n^2)$;
- 2) não, $O(n)$
- 3) $O(\log n)$
- 4) $O(1)$
- 5) não.

CHECKLIST PARA ANALISAR UM ALGORITMO

1. Defina claramente a entrada e seu tamanho.
2. Escolha a operação ou recurso que será contado.
3. Declare o caso: melhor, pior, médio ou amortizado.
4. Conte repetições, chamadas e estruturas auxiliares.
5. Monte uma função de custo quando necessário.
6. Preserve o termo dominante e as variáveis independentes.
7. Informe tempo e espaço separadamente.
8. Meça a implementação se constantes e hardware importarem.

Próxima aula

Makefile